

Relatorio do projeto Banco ClienteServidor

1. Objetivo

O projeto implementa um banco de dados simples em C com arquitetura cliente servidor. O cliente envia operações para o servidor, e o servidor executa as alterações ou consultas.

As operações disponíveis são:

INSERT: insere um registro.
SELECT: consulta um registro pelo ID.
UPDATE: altera o nome de um registro.
DELETE: Remove um registro.
SAIR: encerra as threads servidor.

2. Estrutura

banco.h: define constantes, nomes dos objetos do Windows e as estruturas compartilhadas entre cliente e servidor.
cliente.c: apresenta o menu, recebe dados do usuário e envia requisições.
servidor.c: cria a memória compartilhada, a fila, os semáforos e quatro threads trabalhadoras.
Makefile: define os comandos de compilação.
banco.txt: arquivo criado pelo servidor para persistir os registros.

3. Funcionamento

1. O servidor carrega os registros de banco.txt.
2. O servidor cria uma memória compartilhada no Windows.
3. São criados uma fila circular com capacidade para 10 requisições, semáforos e mutexes.
4. Quatro threads aguardam requisições.
5. O cliente conecta-se aos objetos criados pelo servidor.
6. O cliente coloca uma requisição na fila.
7. Uma thread retira a requisição e executa a operação.
8. INSERT, UPDATE e DELETE salvam as alterações em banco.txt.
9. O cliente recebe e imprime uma resposta.

O acesso à tabela do banco é protegido por `pthread_mutex_t`, evitando que duas threads alterem os dados ao mesmo tempo.

4. Pontos positivos

A separacao entre cliente, servidor e estruturas compartilhadas e clara.

A fila usa o modelo produtorconsumidor.

A capacidade da fila e controlada por semaforos.

A tabela e protegida durante as operacoes concorrentes.

Os dados sao persistidos em arquivo, permitindo recuperar registros apos reiniciar o servidor.

O limite dos nomes recebidos pelo cliente evita ultrapassar o tamanho do campo durante a entrada normal.

5. Problemas e riscos encontrados

5.1 Carregamento do arquivo pode falhar

Em servidor.c, a primeira string de formato usada por fscanf e "%d;49[^\n]\\n". Falta o caractere % antes de 49, portanto o formato nao representa corretamente a leitura do nome. Como consequencia, registros existentes podem nao ser carregados corretamente quando o servidor inicia.

5.2 Canal de resposta unico

Existe apenas uma resposta compartilhada em shm>resp. O servidor usa um mutex para serializar a escrita, mas nao associa a resposta ao cliente que fez a requisicao. Com varios clientes simultaneos, um cliente pode consumir a resposta destinada a outro.

5.3 Encerramento pode ocorrer antes de terminar tarefas

A opcao de desligamento envia quatro requisicoes SAIR, uma para cada thread. Se ainda houver tarefas pendentes na fila, as requisicoes de encerramento podem ser processadas depois ou em conjunto com essas tarefas de forma pouco previsivel. O programa nao possui uma etapa explicita de drenagem da fila antes do encerramento.

5.4 Falta de verificacao de erros em recursos do Windows

O servidor nao verifica se CreateFileMappingA, MapViewOfFile, semaforos, mutexes ou pthread_create falharam. Uma falha nesses recursos pode causar acesso invalido ou travamento.

5.5 Recursos nao sao liberados completamente

O cliente nao desmonta a memoria compartilhada nem fecha os handles abertos. O servidor tambem nao fecha todos os semaforos e mutexes antes de terminar.

5.6 Entrada do usuario pouco validada

As chamadas de `scanf` ignoram varios erros de entrada. Se o usuario digitar texto onde era esperado um numero, o fluxo pode ficar inconsistente. Alem disso, `%49s` nao permite nomes com espacos.

5.7 Portabilidade limitada

O codigo depende de `windows.h`, memoria compartilhada nomeada do Windows e `Pthreads`. Portanto, nao e portavel diretamente para Linux ou macOS.

O Makefile usa `gcc`, `pthread` e `rm`, sendo mais adequado a um ambiente MinGW/MSYS2 do que ao Prompt de Comando comum do Windows.

6. Validacao realizada

A compilacao foi tentada neste ambiente, mas o comando `mingw32make` nao esta instalado ou nao esta disponivel no PATH. Portanto, nao foi possivel confirmar a compilacao executando o Makefile.

A verificacao do codigo foi feita por leitura dos arquivos fonte e do fluxo de sincronizacao implementado.

7. Recomendacoes prioritarias

1. Corrigir o formato de leitura de `banco.txt` e testar carga de nomes com e sem espacos.
2. Redesenhar o canal de respostas para identificar a requisicao e o cliente correspondente.
3. Definir um protocolo de encerramento que espere a fila terminar antes de parar as threads.
4. Verificar o retorno de todas as criacoes e aberturas de recursos.
5. Fechar e desmontar todos os handles e mapas de memoria.
6. Substituir `scanf` por uma leitura validada, especialmente para IDs e opcoes.
7. Documentar o ambiente necessario para compilacao, como MinGW/MSYS2 e a biblioteca `Pthreads` para Windows.

8. Conclusao

O projeto demonstra corretamente os conceitos de cliente servidor, memória compartilhada, fila produtor consumidor, semaforos, mutexes e pool de threads. Para um uso controlado com um cliente, a estrutura tende a atender ao objetivo didatico. Antes de ser considerado robusto, precisa corrigir principalmente o carregamento do arquivo, o gerenciamento das respostas entre varios clientes e o tratamento de erros dos recursos do sistema.